

Your grade: 100%

View Question

Your latest 100% • Your highest 100% • To pass you need at least 70%, the best your highest score.

1. You are designing a Python MCP client that will connect to a local server using STDIO. The client must manage initialization, capability negotiation, and bidirectional communication, while also handling tool invocation and resource reading. Which of the following best describes how this client will interact with the server?

1 / 1 point

- ☐ The client only receives notifications from the server, does not call tools or read resources, and logs messages to a global variable for later analysis.
- ☒ The client spawns the server process, communicates through standard input/output using newline-delimited JSON, negotiates capabilities during initialization, and independently manages tool, resource, and prompt usage.
- ☐ The client connects to the server over HTTP and immediately executes tools without negotiating capabilities or handling messages.
- ☐ The client maps multiple server processes into a single instance and calls all tools on all servers simultaneously without managing separate client lifecycles.

Nice work

STDIO transport involves spawning the server locally, using JSON messages over stdin/stdout, and handling capabilities and operations per the three-phase MCP lifecycle.

2. Which of the following best describes the purpose of the connect method in the MCPClient class?

1 / 1 point

- ☒ Configures how to launch the server subprocess, creates STDIO transport streams, establishes an MCP session with the server, performs the initialization handshake, and ensures proper cleanup via the exit block.
- ☐ Defines the interactive CLI, maps commands to MCP operations, and handles user input and JSON parsing errors.
- ☐ Directly executes tools, reads resources, and retrieves prompts from the server without establishing a session or performing initialization.
- ☐ Closes all connections and resources and terminates the client without establishing communication with the server.

Nice work

The connect method sets up communication with the server, initializes the session, and prints confirmation upon successful connection.

3. You are building an MCP client that needs to connect to a remote production server, perform a file analysis tool, and ensure secure filesystem access. The server may request LLM reasoning during tool execution, and you want to enforce least-privilege file access dynamically. Which combination of MCP features and transports should your client use to implement this scenario correctly?

1 / 1 point

- ☒ Use Streamable HTTP transport for the remote connection, implement sampling to handle LLM requests with user approval, and enforce tools with path validation for secure filesystem access.
- ☐ Use Streamable HTTP transport but disable sampling and Roots, letting the server freely access all filesystem paths while handling LLM requests automatically.
- ☐ Use STDIO transport for local-only execution, implement Roots without path validation, and ignore LLM sampling requests.
- ☐ Use STDIO transport to connect to the remote server, invoke LLM reasoning without user approval, and skip Roots enforcement to simplify file access.

Nice work

Streamable HTTP supports remote bidirectional communication, sampling handles LLM requests securely, and Roots ensures that servers only access authorized directories.

4. You are developing a GQL MCP client using MCPHTTPHandler that inherits from the base HTTP client. The client should allow users to list tools, call them with JSON arguments, access resource templates, read resource content, and retrieve structured prompts. Which of the following statements correctly describes the responsibilities of the GQL-specific methods in this client?

1 / 1 point

- ☐ GQL methods implement the MCP protocol from scratch, including connection handling, JSON-RPC requests, and server capability negotiation, separate from the base class.
- ☐ GQL methods reuse all server responses globally without caching, formatting, and they directly update the server state without using the base client's session management.
- ☒ GQL methods such as get_json_tools, get_call_tool, get_json_resources, and get_prompt wrap the underlying MCP protocol methods, adding input validation, caching, formatting for display, and integration with GUIS, without implementing protocol logic themselves.
- ☐ GQL methods bypass the base client entirely and directly call HTTP endpoints with manual JSON-RPC requests to manage tools, resources, and prompts.

Nice work

The GQL methods adapt the base class protocol methods for GUIs-to-display, handle errors, and ensure proper input/output formatting.

5. You are configuring an MCP client to execute tools safely. The client must decide whether to run, block, or request approval for each tool based on risk and policy.

1 / 1 point

Which of the following correctly describes how permissions and elicitation work in MCP?

- ☐ Permissions and elicitation are optional features that do not affect high-risk tool execution.
- ☐ Permissions run only after the server executes the tool, and elicitation collects data without validation.
- ☒ Permissions are client-side policies (Allow, Deny, Audit) applied before any server interaction, and elicitation is server-initiated structured input that requires user validation.
- ☐ Permissions are stored on the server, and elicitation automatically executes tools without user input.

Nice work

Permissions control execution, and elicitation ensures informed consent with validated input.

6. You are building a base MCP client that enforces permissions and logs actions. The client should check permissions before executing any tool and maintain an audit log.

1 / 1 point

Which of the following correctly describes how the base client handles permission checks and audit logging?

- ☐ The client bypasses permission checks and directly executes tools, logging only errors.
- ☐ The client enforces permissions but does not log any audit entries.
- ☒ The client uses check_permission to determine if a tool call is allowed, denied, or requires approval, and logs every decision with a timestamp using log_audit.
- ☐ The client checks permissions only after the tool executes and logs results retroactively.

Nice work

The base client enforces permissions and maintains an audit log for all operations.

7. You are implementing a GQL client that inherits from the base MCPPermissionClient. Users can select a tool, provide JSON arguments, and execute it via the GQL.

1 / 1 point

Which statement correctly describes how the GQL client integrates permission checking when calling a tool?

- ☐ The GQL client executes tools without extracting arguments or validating JSON, bypassing the base client entirely.
- ☐ The GQL client ignores the base client's permission checks and executes tools directly, managing logs separately.
- ☐ The GQL client only formats JSON arguments and does not handle permission enforcement or audit logging.
- ☒ The GQL method get_call_tool extracts the tool name, params, JSON arguments, and calls call_tool_with_permission from the base client, handling approval prompts and logging automatically.

Nice work

The GQL client leverages the base client's permission checks and audit logging.